

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	5
Bug	Operator-blocked	2
Bug	Queued	5
Bug	Ready for testing	2
Task	In progress	10
Task	Queued	21
Task	Ready for testing	2
TOTAL		47

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at /api/v1/auth/rutacker/captcha then /login once, and the proxy stores the resulting bb_session. The copy paste path and the cookies-file autoload path were both offered and NOT chosen. THE BLOCK NARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest operator_attended boundary). AGENT PREPARED BEFORE THE HAND-OFF, so the opera

ATM ID	Type	Status	Severity	Description
				single attempt succeeds rather than discover broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end, verify the session is actually PERSISTED after successful post (not merely accepted), and verify a stored session is actually USED by a subsequent search. Honest limitation to state up front: a stored bb_session expires, so this path requires repeating — that cost was accepted with the decision. This answer is recorded as consumed DATA per §11.4.35 — it is the operator's state choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applies to decisions). — prior item text follows — RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download proxy + qBitTorrent-go + Jackett + compose environment forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger ongoing coverage-escape backfill
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from mandated test-type matrix
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNKNOWN — INSTRUMENT THE NEXT OCCURRENCE The operator does not currently know which host produces the +0500 Auto-commit fast-forward so the three candidate answers (second Clause session / intentional rsync job / stale job) all remain open. DECIDED ACTION: stop hunting host this session cannot reach (§11.4.6 — remote state is not knowable without access) and instrument the mechanism so the next occurrence identifies itself. Forensic capture
BOB-077	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				add: committer identity, hostname, timezone offset, push timing, and the git remote used, recorded at Auto-commit time into a tracked forensic log. ACCEPTANCE: the next Auto-commit event yields a captured record naming its origin host — at which point this item resolves to one of the three original branches with evidence rather than a guess. This is the §11.4.101 reversible-move: it costs little, blocks nothing, and converts a recurring mystery into a self-identifying event. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5)) bounded-verdict discipline applied to decisions — prior item text follows — RD2-10: Identify second host running the Auto-commit rsync/ssh mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit push script OR retire it
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review the substantive Auto-commit diffs
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items the four Lava-porting findings (closes GA-05)
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)
BOB-090	Task	In progress	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack
BOB-093	Task	In progress	High	RD2-28: Live compose bring-up + verify rutra ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-094	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
BOB-095	Task	In progress	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4 fault injection
				RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for export download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-101	Task	Queued	High	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0 Go --profile go parity remains a goal but is LONGER a v1.0.0 release blocker. RW-10..13 n to a post-1.0 milestone and stop gating the tag Nothing advertised is removed, so §11.4.122 s clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stand unchanged: the Go container's Dockerfile run ONE binary binding only :7187; nothing bind :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md's po map already describes what the container ACTUALLY does and needs no amendment un this decision. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inferen and supersedes any prior agent-chosen defau on this question. Options not chosen are nam above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — GA-19/RW-09: Is --profile go parity a release goal? (gates RW-10..13) — OPERATOR DECISION
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD The tunnel STA LAN-reachable — no bind address changes to 127.0.0.1. The operator explicitly chose to preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regress
BOB-102	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				guard MUST land that FAILS THE BUILD if an LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 THE CONDITION that guard exists — without this decision is unprotected and the item is N closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand maintained list; resolve real auth coverage n grep for the string ‘auth’ (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures p §11.4.107(10). This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — RW-05: LAN-exposure threat mode bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector
BOB-121	Task		Important	

ATM ID	Type	Status	Severity	Description
		Ready for testing		External watchdog for the forced-logout architectural gap (task #85, incident #3)
BOB-135	Bug	Ready for testing	Low	Test isolation: test_list_hooks_after_create fails bulk suite (Permission denied /config)
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc
				OPERATOR DECISION (2026-08-26, §11.4.66): REPAIR, THEN RE-VERIFY UNTIL STABLE The warm ./start.sh path keeps running the ownership repair against a live stack, but waits and re-verifies, and repeats until a pass finds nothing new. Options not chosen: refuse the repair when the stack is up and direct the operator to --recreate; stop the stack on the warm path to (correct but ends 'warm'); accept and document the window. WHY THIS MATTERS MORE THAN A NORMAL ITEM: this is the exact defect the warm 002-user-owned-downloads feature exists to fix. The --recreate path was already fixed to order precondition -> down -> repair -> up so the warm sees a tree no container can write. The warm path was not, so a container can create a new non-owned file AFTER the repair has passed the directory — the ownership problem leaking in through the other door. REQUIRED BY THE CHOSEN OPTION, and it is the hard part: the MUST be BOUNDED, and on giving up it MUST report honestly that ownership is UNPROVEN rather than silently exiting 0 (§11.4.201(6) — loop that stops finding new files because it ran out of iterations returns the same quiet zero genuinely clean tree). An active download creating files faster than the walk completes the realistic non-converging case and must be named in the give-up message. Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options chosen are named so a future reader does not litigate a settled call. — prior item text follows
BOB-159	Bug	Queued	Medium	Reported-Via: §11.4.202 reporting directive 1 on 2026-08-21T19:01:17Z Reported-By: T041

ATM ID	Type	Status	Severity	Description
				independent review (IMPORTANT-2), partially remediated What (the report, verbatim): The <code>–recreate</code> path was fixed this round: <code>run_ownership_precondition -> stack_down</code> <code>run_ownership_repair -> stack_up</code>, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by the new checks in <code>tests/unit/test_start_reload_recreate.sh</code>, each killed by a paired 1.1 mutation (11/11 RED-OK). The <code>WALK</code> path is NOT covered by that fix and this item tracks the remainder. On a warm <code>./start.sh</code> the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records complete over a time that is not, and those stragglers are never repaired without a manual <code>–force</code>. BOUNDED dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits without walking (<code>marker_is_valid</code>) so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-only probe mode on <code>scripts/ownership_repair.sh</code> that answers has-this-scope-been-repaired without walking the tree. The existing <code>–dry-run</code> cannot serve: it deliberately SKIPS the marker fast-path (<code>FORCE=0 && DRY_RUN=0</code> guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding that mode requires editing <code>ownership_repair.sh</code>, which another stream was actively editing during this round; duplicating <code>marker_is_valid</code> into <code>start.sh</code> instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misleading comment at the warm-start call site (which

ATM ID	Type	Status	Severity	Description
BOB-162	Task	Queued	Medium	claimed the gate runs before any container writes) has been corrected to state this bound honestly. Affected scope / file-scope manifest start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path) Reproduction / context: 1. Ensure the stack UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write to it. Acceptance criteria: A warm ./start.sh can complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal naming –recreate, or the stack is quiet for the walk. Asserted behaviourally in tests/test_start_reload_recreate.sh alongside the existing PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with a paired 1.1 mutation that fails it. Reported-Via: §11.4.202 reporting directive + on 2026-08-21T19:35:02Z Reported-By: BOB-107 remediation, residual What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build also cover tests/hooks. MEASURED before/after the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 3 with 3 under tests/hooks. All three pass. But the GUARDS themselves are still invoked by nothing check-brief-inputs.sh is honestly conductor-rather than automatic: a brief's required-input list lives in free-form prose that the Agent too structured input does not expose, so an automatic prompt-scraping gate would itself be an unproven pattern-match. Its seam is the dispatch procedure, and that is a documented

ATM ID	Type	Status	Severity	Description
				+ habit change, not a hook. unattributed-commit-guard.sh is different and is why this item exists. Run against real history it names 14 violating commits since the last tag (and 21 bare Auto-commit subjects exist across all refs). Wiring into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, pre-existing debt nobody can fix in the moment. That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until 14 are attributed; (b) one-time monotone-decrease ratchet (the 11.4.135 pattern) – snap 14 as the baseline, the count may fall and may never rise, so day one is green and the disease cannot spread; (c) changed-commits-only – good only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate. Recommendation, with the reasoning rather than just the pick: (b). It is the pattern this constituent already uses for exactly this situation, it makes the debt visible and bounded immediately, and cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's call and this item is not closed until that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their ATTRIBUTION is absent and that nothing currently prevents the 15th. Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh; scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, especially proven non-bluff against a no-op stub). Neither is invoked by scripts/pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts

ATM ID	Type	Status	Severity	Description
				standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage). Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing to this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an inverted ratchet. WHAT. The BOB-109 scaling suite gates its two timing tests on $\text{loadavg_1m/nproc} \leq 0.75$ — on this 8-cpu host, a 1-minute load average at or below 6.0 — and SKIPS loudly otherwise. The gate is correct and was adopted for a good reason (§11.4.201(8)): the author validated a 2.10 growth-exponent threshold, then observed it FALSE-FAIL correct code at load 11.06 (baseline span 2.136), and at load 15-23 measured baseline spans of 2.358 against injected-cubic mutants 2.278-2.331 — the signal is smaller than the noise and the two are not separable in either direction. Gating was the honest response. THE PROBLEM. The gate has consequently NEVER RUN under its own precondition. Every recorded run today measured load 9.15-23; the independent reviewer measured 9.66 during review; the conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at loads 9.4-11.6 (span 1.882) and in three stability runs (1.759/1.832/1.848) — but never once beneath the shipped ceiling. The author states this honestly as an inference from data rather than a run the author can paste, and the reviewer confirmed the inference is sound (quiescence is strictly more favourable). Sound inference is still not an observed run. WHY THAT MATTERS (§11.4.226). A registered, topology-present guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled quiet-window invocation, no tracked item —
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				which is precisely the unexecuted-standing-guard class §11.4.226 names, where registrat gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is two standing guards that, when finished, immediately emitted latent FAILS. ACCEPTANCE. (a) Execute both quiescence-gate timing tests on a host whose 1-minute loadavg genuinely $\leq 0.75/\text{cpu}$ — no agent fleet, no concurrent build — and capture the run as an artifact showing the resolved load reading alongside the measured span exponent, so that precondition is provable from the artifact and not asserted. (b) Record the observed GREEN polarity in the item and in the test source, replacing the current honest-inference note. (c) If the gate FAILS under genuine quiescence, then the finding this item exists to surface, and it reopens the threshold-calibration question rather than being worked around. (d) Establish freshness contract per §11.4.226: how stale a quiescent verdict may be before the gate could be as uncovered again, so this does not silently return to never-executing. HONEST BOUNDARY. This item does not claim the gate is wrong. The evidence points the other way — quiescence is strictly more favourable than the loads where GREEN was already observed. It claims only that the run has not happened, and that an unexecuted guard cannot be cited as coverage. PROVENANCE. Raised as IMPORTANT-2 by the independent Fable-substrate reviewer of the BOB-109 work, and independently corroborated by the reviewer verified the SKIP is loud and precise, its resolved numbers, and confirmed the ship tests do SKIP at today's load. WHAT. The rutracker SEARCH endpoint is refused by bot protection while the site itself is fully reachable. Measured unauthenticated 2026-08-21 (no cookie file read, no credential value in any artifact — §11.4.10): GET /forum/index.php -> 200, 96283 B,
BOB-172	Bug	Ready for testing	High	

ATM ID	Type	Status	Severity	Description
BOB-175	Task	Queued	Low	WHAT. The workable-items update seam guards the status-location invariant in one direction only. After BOB-166, update --status <terminal> on an Issues-located row is refused. The mirror is still open: update --location Fixed --status <non-terminal> exits 0 and creates a row that the update's own validator immediately rejects. Verified empirically by the independent review of BOB-166: update --location Fixed --status 'In progress' -> exit 0 validate -> FAILS, naming the row (check (f), fixedLocationNonTerminalState). So the command can still mint a state its own validate call refuses. That is the §11.4.196(F) shape at the seam layer: the invariant is CONFIGURED in validate but not ENFORCED on every write path that can violate it. WHY IT WAS DELIBERATELY LEFT OPEN IN BOB-166, and that was right. Three reasons, all checked rather than asserted: (1) the real corpus has ZERO instances of this direction against TEN of the direction BOB-166 closed — the asymmetry in the data matched the asymmetry in the code; (2) the detective coverage ALREADY existed and demonstrably fires, so unlike BOB-166's direction this cannot rot silently; (3) and the load-bearing one — reopenCmd documents and SANCTIONS a transient Fixed-plus-non-terminal state via its location Fixed operator override. A naive preventive guard at the update seam would collide with that override's semantics. Closing this therefore requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined. ACCEPTANCE. (a) Decide and RECORD how a preventive guard on this direction coexists with reopenCmd's sanctioned override — this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidates worth weighing: exempt the reopen path explicitly, require an override flag on update mirroring reopen's, or leave the

ATM ID	Type	Status	Severity	Description
				direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it reuses terminalStatuses() — BOB-166 established a predicate source specifically so the two directions cannot drift. (c) Paired §1.1 mutation (d) NEGATIVE CONTROL (§11.4.201(1)), and it's the sharp one here: the sanctioned reopen path MUST still work. A guard that breaks reopenCmd's documented override is a false-positive refusal, and worse than the gap it closes because it breaks a workflow operators are told to use. HONEST BOUNDARY. This is not a live data defect. Zero corpus instances, and the detective gate catches it at the next validate. It's filed so a scoped, deliberate omission stays visible as a tracked decision instead of living in a code comment where the next reader will mistake it for an oversight (§11.4.197: a started thread reaches a documented terminal state, never quiet abandonment). PROVENANCE. Raised by the implementing agent of BOB-166, a known asymmetry it deliberately did not close independently verified and endorsed as an acceptable scope boundary by that item's reviewer, on the condition that it become a tracked item rather than a comment. This is the item. Three icon-glyph controls (.bridge-retry, .then toggle, .caret) render text made of non-BMP code points. axe-core declines to decide their control reporting them under the incomplete channel with messageKey nonBmp and resolving NEITHER fgColor NOR bgColor, so the Python recomputation in docs/qa/BOB-164/axe_contrast_scan.py cannot decide either: 40 such nodes across 16 palette x mode combinations. BOB-164 round 2 stopped them from being scored as silent passes by naming them GLYPH_UNMEASURED, printing them on every run and fencing them, so any NEW unmeasured glyph node blocks. What remains genuinely
BOB-184	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				unproven is their contrast itself: as non-text interface components they owe 3:1 under WCAG SC 1.4.11, and no oracle in this repo measures that today. Acceptance: a measurement path resolves the effective foreground and backdrop for glyph controls (for example reading <code>getComputedStyle</code> colour against the composited backdrop, or replacing the glyphs with inline carrying explicit fill tokens), each of the three controls shown to clear 3:1 in all 16 palette x mode combinations, and the <code>GLYPH_UNMEASURED</code> fence shrunk by exactly the nodes then proven. <code>docs/qa/BOB-164/axe_contrast_scan.py</code> scans static dist with no backend, so every node whose existence depends on fetched data is absent from the page it measures: the results table renders empty, so <code>.type-badge</code> and <code>.quality-badge</code> never appear, and the hooks list renders empty, so <code>.status</code> pills never appear. That is precisely why the BOB-164 round-1 regression went unseen in a rendered DOM even though it was scanned in all 16 themes, and it is why round-2 had to close the class in the arithmetic oracle instead. Those nodes are now covered by <code>frontend/src/app/models/style-contrast.spec.ts</code> which extracts declared foreground-on-fill pairs from the stylesheets, but that is an ARITHMETIC claim about declared colour pairs, not a rendered-pixel one: it cannot see opacity applied by an ancestor, a composited backdrop, or a cascade this repo's static extractor does not model. Acceptance: the scan drives the dashboard with seeded fixture data (a stub backend, a route fixture, or an injected component harness) so the badge and status nodes are present in the DOM, axe measures them directly, and a control needle proves the rendered oracle sees a seeded sub-floor badge before any clean result from it is believed.
BOB-185	Task	Queued	Medium	<code>scripts/ownership_precondition.sh</code> consumes SAME unreviewed <code>.env</code>-driven scope as
BOB-187	Bug	In progress	—	

ATM ID	Type	Status	Severity	Description
BOB-188	Bug	In progress	—	ownership_repair.sh and creates probe files inside it, but it is NOT fenced. T028 remediation added ownership_path_fence() to scripts/lib/ownership.sh and wired it into ownership_repair.sh, closing the path-escape class there; the precondition was outside the agents file scope and still accepts whatever config/owned_paths.yaml expands to. Blast radius is lower than a recursive chown (it writes probe files rather than mutating ownership of existing tree) but the INPUT is identical and equally unreviewed, so the same QBITTORRENT_DATA_DIR value that would have driven a filesystem-wide chown will drive probe file creation at an arbitrary location. Note the escape is empirically confirmed for the repair path, not merely reasoned: during T028 remediation a probe declaring the verbatim shipped entry with QBITTORRENT_DATA_DIR hung the suite, and ps showed the real artifact executing “find / (! -uid 1000 -o ! -gid 1000) -printf ...” before it was killed by verified pid. Acceptance: ownership_precondition.sh calls SAME ownership_path_fence() predicate from scripts/lib/ownership.sh (never a second dial 11.4.251), refuses fail-closed on a rejected entry and ships a paired 1.1 mutation proving the refusal fires PLUS a negative control proving six live shipped entries still ACCEPT so the fix is not a 11.4.201(1) false-positive refusal. Discovered out-of-band during T028 remediation and reported by the agent as needing tracking; it is also a 11.4.238 coverage escape. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNTRACK THE BINARY — BUILD ON DEMAND §11.4.30 and recorded as consumer DATA: the compiled workable-items binary is NOT to be git-tracked. ‘Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned’ applies without exception here. Keeping it tracked with only

ATM ID	Type	Status	Severity	Description
				invariant-52 fingerprint gate, and keeping it tracked with a commit-time rebuild, were both offered and NOT chosen. REQUIRED END STATE: the gate builds from source, or REFUSES HONESTLY when the Go toolchain is absent (§11.4.201(11) — probe the artifact through its real invocation path, never fake a pass, never silently fall back to a stale binary). Staleness becomes structurally impossible rather than merely detected. ACCEPTED COST: a fresh clone needs a Go toolchain before invariant 17 can be — already true for the qBitTorrent-go backends so this adds no new host requirement. The invariant-52 fingerprint gate remains useful for defence-in-depth for as long as any binary exists on disk. This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applies to decisions). — prior item text follows — pre_build_verification.sh invariant 17 (CM-WORKABLE-ITEMS-VALIDATE) resolves its binary through the candidate loop at :534, where the FIRST entry is constitution/scripts/workable-items/bin/workable-items. That file is GIT-TRACKED (md5 17644a248363, identical to bin/workable-items-linux) and executable, so it VETOES resolution over the current untracked sibling constitution/scripts/workable-items/workable-items (md5 43376a6d0184). The tracked binary is STALE: it does not contain the guards its own source now has. MEASURED, needle-proven, 2026-08-25. String presence in the tracked binary “refusing to set terminal status” -> 0; “Issues- location item has TERMINAL status” -> 0; compare needle from the SAME updateCmd, “at least one mutable field flag is required” -> 2 (so the instrument sees through that path and the zeroes are real absences, not a blind read). The untracked sibling returns 1 / 1 / 2 for the same

ATM ID	Type	Status	Severity	Description
				three queries. A check whose message string absent from the binary cannot fire. RUNTIME PROOF: the same injected violation (a row set terminal status while current_location=Issue run through both binaries -> stale reports 1 violation (only the body_md desync class), current reports 2 including the location-status class verbatim. This is exactly why the ten BOB-166 rows (BOB-087/129/131/144/145/146/148/153/155/156) survived undetected: the gate that was supposed to catch them was running a binary structural incapable of seeing them. The 11.4.108 SOURCE >ARTIFACT gap: source correct, artifact stale, every gate reading the artifact reports green. This is the THIRD instance of that class found one session, alongside BOB-169 (a charset gate that never opened a file) and BOB-183 (a service bundle five days older than its sources). COMPOUNDING: the comment at :519-523 directly above the loop asserts “The naive bin/workable-items path never existed in this checkout (bin/ is a gitignored local build-output dir nothing ever populated)”. That statement is now FALSE - bin/ holds two tracked binaries. stale comment asserting the absence of the v file that now shadows resolution is how this stayed invisible. Also an 11.4.30 question the operator owns: bin/workable-items and bin/workable-items-linux are VERSIONED BUILD ARTIFACTS in a constitution submodule that inherited by reference by every consumer, so every consumer inherits whichever binary was last committed. Flagged independently by two separate agents this session. ACCEPTANCE: (a) resolution must never prefer a stale artifact over a current one - either the tracked binaries are removed and resolution falls through to build on-demand, or a freshness check refuses a binary older than its sources (see the BOB-183 fingerprint gate for a working pattern); (b) the false comment at :519-523 corrected; (c) a pair 1.1 mutation proving the new refusal fires, p

ATM ID	Type	Status	Severity	Description
				negative control proving a genuinely fresh binary still passes so the fix is not an 11.4.201 false-positive refusal; (d) the 11.4.30 tracked-binary decision recorded as operator DATA. Discovered out-of-band during BOB-166 remediation - a 11.4.238 coverage escape in its own right. WHAT: the §11.4.252 fail-open scan reports 0 for qBitTorrent-go, and that zero is NOT evidence. The triage agent ran control needle through the scanner's own path per §11.4.203 (b): a Python 'except Exception: pass' needle was SEEN, a TypeScript 'catch (e) {}' needle was SEEN (so frontend/src's zero IS a real zero), but a Go empty-'if err != nil {}' needle was NOT SEEN. The instrument that cannot see the idiom returns the same quiet zero as a clean tree, and only one of those is honest. DISTINCT FROM BOB-189, BOB-189 is a false MATCH (fail-closed guards reported as fail-open); this is a false NULL (an entire language unscanned). Same scanner, opposite failure directions, different fixes - merging them would hide one behind the other. IMPACT: Go is the language of qbittorrent-prg and boba-jackett (port 7189, which owns encrypted tracker credentials), so the unscanned surface is exactly where §11.4.252's credentials-plus-mutation combination is most likely. Nobody has assessed it; the dashboard says clean. ACCEPTANCE: the scanner grows a Go covering the empty-err-block and swallowed error idioms, its needle is SEEN through the path, qBitTorrent-go's result is re-derived, and every finding is triaged as this Python/TS pass was. Until then qBitTorrent-go's fail-open position is UNKNOWN and must be reported as UNKNOWN, never as 0. === ROUND-2 INVESTIGATION, MEASURED 2026-08-26 — THE FILED TITLE WAS TOO NARROW (§11.4.6) == The live scanner is constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh,
BOB-191	Bug	Queued	—	

ATM ID	Type	Status	Severity	Description
				driven per-root by invariant 39 at scripts/pre_build_verification.sh:1481-1544. (scripts/pre_build/check_cm_no_fail_open_skip.sh has NEVER existed on any branch — it exists only on an uncommitted worktree. Any citation of the path is citing a file that is not there.) IT IS A WEAK-MATCHER DEFECT, NOT A SCOPE HOLE Line :243 sets exts="py go rs c cc cpp h hpp java cs js ts jsx tsx php rb" — .go IS enumerated. By line :537 gates the Python ast analyser on cases "\$f" in *.py), so every non-Python file falls through to just two text matchers (:772 empty catch (...) {, :794 credential = x \\ "literal"). Go has no catch keyword, so neither matcher CAN fire on Go by construction. NEED TEST, BOTH ARMS (planted / rc / result): Python 2 / 1 / SEEN · TypeScript 1 / 1 / SEEN · Java 1 / 1 / SEEN · Go 6 / 0 / NOT SEEN — PASS · Rust 2 / 0 / NOT SEEN · Ruby 1 / 0 / NOT SEEN · C 2 / 0 / NOT SEEN. Go is ONE OF FIVE BLIND EXTENSIONS, not a special case. The Go needle is real code: build rc=0, go vet rc=0, gofmt -e clean. THE DECISIVE DISCRIMINATOR — same bytes, extension changed: .zig (NOT enumerated) → "SKIP — topology_unsupported"; .go (enumerated) → "PASS". The gate ALREADY OWNS an honest-blindness mechanism and it works correctly; Go routes around it precisely BEING ENUMERATED. A scope hole announced itself; this matcher hole prints green. That makes it the worse defect and is why the corrected test leads with it. QUANTIFIED SURFACE (exclusion stated, §11.4.224(E)): using the scanner's OWNED prune list and nothing added — 106 .go files / 20,389 LOC (52 production files / 6,858 LOC). WITH and WITHOUT exclusions the count is == 106; no vendored tree exists, so no unstateful exclusion could manufacture a clean number qBitTorrent-go holds 106 blind files and 0 analysable files — 100% blind — yet invariant counts it among "5 first-party source root(s)" reported clean. DEFECTS THE BLINDNESS HOLE filed as BOB-204 (credential DELETE returns

ATM ID	Type	Status	Severity	Description
				unconditional 204 while the .env plaintext de error is discarded; plus the env_write_failed_db_rolled_back code asserti an unconfirmed rollback). SEPARATE SCOPE HOLE → filed as BOB-205 (cmd/boba-ctl, 947 I orchestrator, absent from DANGER_ROOTS entirely). Distinct-but-similar per §11.4.214, deliberately not merged. GOLDEN-FALSE FIXTURE FOR ANY FUTURE GO ARM: interna jackettapi/auth_middleware.go:45-66 is corre fail-CLOSED (constant-time compare, 401+ret — a Go arm MUST NOT flag it. LOAD-BEARIN §11.4.250 PREDICTION: bolting a naive Go reg arm on will IMMEDIATELY manufacture BOB-189-class false positives in Go — if err nil { return false } in a Go validator is fail CLOSED. Fixing this as layer N+1 reproduces BOB-189 in a new language. THEREFORE the remediation order is fixed: (1) analyser REGISTRY with an UNANALYSED verdict FIR extension→analyser map, unmapped extensi reports UNANALYSED never silent PASS, whi converts the false null into an honest gap and covers rust/ruby/C too; (2) only then a Go arm call-site-aware from day one (go/ast, or adopt errcheck/staticcheck rather than hand-rolled regex), shipped with the auth_middleware golden-FALSE. ONE PRIMITIVE, TWO SYMPTOMS: the scanner classifies by local syntactic shape and never consults semantic — BOB-189 ignores the CALLER’s context (fal match), BOB-191 searches for a shape from a DIFFERENT language family (false null). The gate’s own header already concedes the prim (“requires real data-flow analysis this gate CANNOT honestly claim”); what is missing is propagating that concession into PER-LANGU honesty. Keep two tracker items. INSTRUMEN BUG RECORDED, NOT HIDDEN (§11.4.201(12) the investigation’s first verdict extraction use grep -oE 'PASS\ FAIL\ SKIP', which matched FAIL inside the gate’s own name (...FAIL-CLO and mislabelled three PASSes. Re-run keyed c

ATM ID	Type	Status	Severity	Description
				exit code; the table above is the corrected run EVIDENCE: docs/qa/BOB-191/ scanner_blindness_investigation_20260826.m (255 lines, §11.4.44 header). No source, gate, c config modified by the investigation. === ROUND-3 PARTIAL FIX, MEASURED 2026-09-2 STEP (1) OF THE ROUND-2 REMEDIATION OR LANDED; STEP (2) STILL OPEN, ACCEPTANCE NOT YET MET === Per the round-2 remediati order, step (1) — the analyser REGISTRY with honest UNANALYSED verdict — is now implemented and independently re-verified (conductor re-ran the mutation suite from a clean shell, 206/206 green). Step (2) — a real semantic Go arm — is DELIBERATELY DEFERRED, so this item stays Queued: the ACCEPTANCE criterion above (“its needle is S through the real path”) is still unmet for Go/F Ruby/C. CORRECTION TO ROUND-2’s OWN FIGURE (§11.4.6, re-verified live, not re-guess round-2 said “5 blind extensions (go/rs/rb/c/h Re-run live against all candidates this round: (.cpp, a genuine empty catch() {}) is SEEN — rc=1 FAIL, already correctly caught. It was ne actually blind; .h is content-ambiguous (same header extension serves both C and C++) and not a distinct 5th case. Exactly 4 languages a structurally blind: Go, Rust, Ruby, C. This narrows, not widens, the round-2 figure. WH LANDED (constitution/scripts/gates/ cm_dangerous_combination_fail_closed.sh — constitution-submodule file, inherited by reference; landing it for real still needs the submodule’s own §11.4.26 fetch/pull/push workflow, separate from boba’s commit path New DANGEROUS_COMBO_UNANALYSED_EXT registry (default go rs rb c). Verdict now has 3 bran real hits → FAIL (unchanged); zero hits + UNANALYSED files present → new honest △ M – N file(s) UNANALYSED... + PASS (PARTIAL COVERAGE) (still exit 0, advisory — never block build on an unanalysed language); zero hits - zero unanalysed → unchanged original clean

ATM ID	Type	Status	Severity	Description
				PASS text. - scripts/pre_build_verification. invariant 39 (boba-owned): fixed to read the UNANALYSED count unconditionally, not only nonzero exit — otherwise a 100%-UNANALYSED root would still have silently printed “no fail- open anti-pattern across N clean roots” with caller never seeing the NOTE. - Mutation test extended with 17 new assertions covering Go Rust/Ruby/C UNANALYSED-branch RED/GREEN C++ negative control (still caught, never marked UNANALYSED — the registry must not swallow working extension), a clean-root negative control, and a mixed-real-hit-not-masked control. Full suite 206/206 (189 pre-existing + 17 new) independently re-run by the conductor from a clean shell. LIVE FULL-REPO RESCAN RESULTS (real DANGER_ROOTS, this round): Python backlog unchanged at 59 hits (download-problem- src:25, plugins:32, scripts:2 — pre-existing, out- scope, advisory since 2026-08-20). New honeypot signal: 121 files now reported UNANALYSED (qBitTorrent-go:117, cmd/boba-ctl:4 — BOB-2026- scope hole now visibly folded into the same honest-gap mechanism) instead of silently counting toward “clean.” Zero new FAIL findings in Go — by design; a real Go checker needs ca- site-aware semantic analysis, still deferred. VULNER- STEP (2) STAYS DEFERRED, NOT A REGRESSION. the round-2 §11.4.250 PREDICTION stands unrefuted — a naive Go regex arm would immediately flag internal/jackettapi/ auth_middleware.go:45-66 (the golden-FALSE- fixture) as a BOB-189-class false positive. A re- arm needs go/ast or an adopted tool (errcheck/ staticcheck), shipped WITH that golden-FALSE- passing, before it can fire. REMAINING WORK CLOSE THIS ITEM: implement the Go semantic arm (go/ast-based or errcheck/staticcheck- adopted) covering the empty-err-block and swallowed-error idioms, verified against the auth_middleware.go golden-FALSE fixture, re- derive qBitTorrent-go’s real result (currently UNKNOWN, honestly reported as such), and

ATM ID	Type	Status	Severity	Description
BOB-194	Bug	In progress	—	trriage every real finding as the Python/TS pass already is. Rust/Ruby/C have zero first-party findings in boba's tree today (verified this round) so the own arms are lower priority until such files emerge — the UNANALYSED registry already covers them honestly in the meantime. EVIDENCE: closure_evidence_20260925.md (RED/GREEN transcripts, the C++ correction with live proof, the live full-repo scan table, anti-blob provenance). State left uncommitted by design (constitution-submodule files need the §11.4.201 workflow, not boba's commit-push-all.sh). WHAT: agent worktrees under .claude/worktrees/ are ephemeral copies pinned at arbitrary commits - never built, never shipped, deleted when the agent finishes. Pre-build gates that walk the tree with find(1) from the repo descend into them anyway. MEASURED INSTANCE (2026-08-25): CM-GO-TOOLCHAIN-MATCHES-BUILDER FAILED the main build with exactly one finding, from .claude/worktrees/agent-afda1df906c537ab4 - a worktree 86 commits behind still carrying 'FROM golang:alpine' against a go.mod long since at 1.26.2. The main tree was CORRECT throughout. Per §11.4.201(1) a false-positive refusal is a FAILURE of equal severity to a false pass: it blocks real work and teaches people to bypass the gate. FIXED FOR ONE GATE: '.claude' added to that gate's PRUNE_DIRS with a deterministic §1.1.1 - a stale mismatch inside .claude/worktrees NOT fail a healthy tree, the SAME mismatch outside it MUST still fail, so the prune cannot widen into blindness (§11.4.201(6)). Mutation verified: reverting the prune fails 2 checks. SURVEY, measured not inferred: 8 of 13 pre_build gates walk the tree; only the fixed one prunes .claude. Gates using 'git ls-files' are structurally immune - worktrees are untracked. Empirically probed: check_cm_closure_seam_binds, check_cm_killpg_pgid_guard and

ATM ID	Type	Status	Severity	Description
BOB-196	Task	Queued	—	check_cm_no_production_mutation_residue . rc=0 with ZERO worktree references - clean i practice today. UNMEASURED, not clean: check_cm_test_mock_pid_explicit_int and check_cm_test_mock_pid_patched_when_rea were not reached before the probe budget expired. HYPOTHESIS REFUTED, recorded so nobody re-walks it: an earlier revision of this item speculated that check_cm_plugin_count 90s timeout was caused by walking worktree is NOT. That gate's find is scoped to \$PLUGINS_DIR (check_cm_plugin_count.sh:2 so worktrees are outside its walk entirely. Its slowness is real - still running past 54s on a r measure - but has a different, still-unidentifie cause and belongs to its own item, not this or ACCEPTANCE: every tree-walking gate either prunes agent scratch or is proven immune by measurement, each with a paired mutation. NOTE the asymmetry that bounds the risk: a worktree can only ADD findings to a security relevant scan (killpg, mutation-residue), neve remove them - so the exposure is false refusal and eroded trust, not a missed defect. WHAT: the CM-PLUGIN-COUNT pre-build gate takes 189 SECONDS to verify 8 documented counts. Measured 2026-08-25: exceeded a 90s probe budget (rc=124), then a bounded re-run completed at 'rc=0 wall=189s' - it PASSES, it is simply slow. CAUSE, MEASURED NOT GUESS the marker loop at check_cm_plugin_count.sh:273-290 iterates EVERY LINE of every governed document and spawns a pipeline per line. Per iteration: 'pri grep -oE wc -l tr -d' (4 processes) plus 'pri sed -n' (2) = ~6. Governed docs are AGENTS.m 689 + CLAUDE.md 494 + docs/features/Status. 670 = 1853 lines, so ~11,118 process spawns, measuring ~102 ms/line. Classic per-line-subprocess shell trap. THE IRONY WORTH PRESERVING, because it explains why nobody simplified it: the in-code comment shows 'oE

ATM ID	Type	Status	Severity	Description
				wc -l' was chosen DELIBERATELY over 'grep -o' after measuring the §11.4.201(12) footgun - on this host (ugrep 7.8.4) 'grep -coE' returned 3 at top level but 1 inside a 'set -euo pipefail' subshell, a context-dependent reading. The correctness of the gate is right and must be kept; it is the PER-LINE application of it that costs the three minutes. HYPOTHESIS REFUTED, recorded so nobody walks it: this is NOT worktree traversal. The gate's find is scoped to \$PLUGINS_DIR (:224), the five agent worktrees are outside its walk. That guess was raised on BOB-194 and is dead. LIKELY FIX (unverified, stated as a candidate for a conclusion): pre-filter with ONE grep for lines containing 'CM-PLUGIN-COUNT:' before entering the loop, reducing ~1853 iterations to the ~8 lines that carry a marker, while leaving the per-line parsing logic - and its footgun-avoiding form - completely unchanged. WHY IT MATTERS beyond impatience: this gate is on the pre-build critical path, and CLAUDE.md leans on it as the mechanical authority for the three distinct plugin rosters (43 curated / 43 engines / 12 bootstrap) whose conflation was BOB-149. A gate slow enough to tempt anyone into skipping it protects nothing. ACCEPTANCE: runtime reduction with the counts and the footgun-avoiding pattern preserved byte-for-byte in behaviour, proven the existing paired mutation still biting plus a before/after wall-clock recorded. §11.4.6: not here questions the gate's VERDICTS - only its runtime. STATUS: LATENT on this host (measured: all declared locations sit on ONE btrfs mount, full ownership-expressive) and OPERATOR-GATE for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount is a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being exFAT or NTFS is entirely ordinary. TWO RESIDUALS: (1) REPAIR-LAYER
BOB-211	Bug	Queued	minor	

ATM ID	Type	Status	Severity	Description
				DIAGNOSIS. On a uid-flattening mount whose flattened uid is NOT the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 both refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstype-aware — so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, Not proven: the shim used to verify BOB-206 mode the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dm so the preserve_mode: true / 600 contract on and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak the mode check cannot see. WHY OPERATOR-GATED: a genuine end-to-end RED needs a real uid-flattening mount. The privilege-free route is bindfs -u (FUSE; fusermount IS present and user_allow_other IS set) — but bindfs is ABSENT on this host, as are mkfs.vfat and losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was attempted (§11.4.21 — high-blast-radius host mutation is operator-gated). ALSO RECORDED NO test anywhere exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure across the three ownership test files, needle-proven (probe_location = 3 hits through the shim instrument). ACCEPTANCE: (1) operator decides whether to install bindfs or provide a privileged loopback so the RED can be built; (2) if verified the repair path gains fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mount and refuses, or documents honestly that it cannot be enforced there. DISCOVERY CHANNEL

ATM ID	Type	Status	Severity	Description
BOB-219	Bug	In progress	major	(§11.4.238): retained residual from the BOB-2 verification stream. WHAT, conductor-verified: docs/guides/tracker-credentials.md is TRACKED. Its §11.4.65-mandated export twins docs/guides/tracker-credentials.html and .pdf both EXIST ON DISK but are NOT tracked, and ARE ignored — matched by the .gitignore <i>credentials</i> deny-all at :31. The .pdf is rescued by an explicit allowlist entry at :38. Nobody added entries for the twins. WHY IT IS A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in-scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.212 requires every §11.4.65-scope doc to be reachable from the README. Two artifacts that exist locally but can never be committed are, from any fresh clone, ABSENT: the tracker-credentials.html and .pdf, so every clone of this repository has a tracker-credentials guide with no exports, while the authoring host looks complete. The divergence is invisible on the machine that would notice it. WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL of the BOB-212 class. git status never listed the twins, so no gate and no author had a signal. The blind instrument and the clone tree return the identical quiet zero. ANOTHER INSTANCE OF THE SAME MECHANISM, filed in qa/BOB-124/evidence/loginctl_user_state.txt instead of separately because it is one fix: docs/qa/BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and is therefore uncommitted §11.4.83 QA evidence SIBLINGS IN THE SAME DIRECTORY — user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log — ARE tracked, saved only by the robust glob negation !docs/qa/**/. That one file is lost purely because it is .txt rather than .log. The pattern is exact: robust glob negations hold; per-file and per-extension rescues leak. ACCEPTANCE: (1) both twins become trackable and are committed along with their .md; (2) the BOB-124 .txt evidence likewise

ATM ID	Type	Status	Severity	Description
				(3) whichever BOB-212 fix direction is chosen MUST cover these — a fix that closes the future swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check that a tracked .md in §11.4.65 scope has COMMITTABLE twins, so this class cannot recover silently. DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep — and only on its SECOND pass. The first pass filtered by a source-extension set that omitted .pdf, which closed this finding entirely; the agent re-ran with no extension filter over all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowlist is itself a false-null generator, which has the same shape as the defect being investigated. === ROUND-2 PARTIAL CLOSURE, MEASURED 2026-09-25 — criterion 1 already done, criterion 2 unsatisfiable, criteria 3+4 done ONLY for the .md-export-twin class === Independently verified via <code>git check-ignore -v</code> on all three named artifacts: CRITERION 1 (tracker-credentials.{html,pdf} twins committable) — ALREADY DONE, pre-dating this round: commit 9f6f517 already added !docs/guides/tracker-credentials.{html,pdf} rescue lines (.gitignore:49-54); both are tracked and clean. Nothing to do. CRITERION 2 (BOB-124 loginctl_user_state.txt trackable) — CLOSED AND UNSATISFIABLE. The file does not exist on disk and has ZERO trace in <pre>git log --all --diff-filter=A -- "*loginctl_user_state*", and is absent from the entire working tree. Its .log siblings in the same directory ARE tracked (rescued by the existing docs/qa/**/*.*.log negation), confirming the file was lost purely by having a .txt extension, exactly as the original report claimed — but there is no recovery path and no legitimate way to “make it trackable” without fabricating QA evidence, which is itself an anti-bluff violation. This criterion is permanently closed, not deferred</pre>

ATM ID	Type	Status	Severity	Description
				CRITERION 3 (general mechanism preventing recurrence) — PARTIALLY closed. A new guarantee (see criterion 4) closes the class criterion 1 exemplified: a TRACKED .md file whose §11.4 export twin is gitignore-swallowed. It does NOT close the class criterion 2 exemplified: a NON-.md QA-evidence file (any extension) swallowed by an unrelated gitignore pattern. Extending the existing !docs/qa/**/*.*.log negation to also cover .txt was concretely measured this round and PROVEN UNSAFE for this project: .gitignore documents cookies_* as this repo's own secret-file naming convention and a scratch-tree experiment confirmed a .txt-wide rescue would also un-ignore cookies_rutacker.txt / RUTRACKER_session.txt my_password.txt — real credential-shaped files already relying on that exact pattern staying broad. This candidate fix is now closed off with evidence rather than left for a future session re-attempt and re-measure. THE REMAINING GAP: a general mechanism for “a QA-evidence file legitimately meant to be tracked gets silently swallowed by a broad secret-shaped-filename gitignore pattern, distinguished from an actual secret file” remains genuinely open — it needs a smarter signal than file extension (e.g. content sniffing, or an explicit per-file allowlist-on-creation discipline) that this round did not attempt to design, since inventing one hastily risked exactly the credential-leak class the constraint exists to prevent. CRITERION 4 (a check exists) — DONE for the .md-export-twins class. New gate scripts/pre_build/check_md_export_twins_committable.sh + self-validated test tests/pre_build/test_check_md_export_twins_committable.sh arms: RED, GREEN, 2 golden-FALSE, fail-close x3, real-repo run, §1.1 paired mutation — all independently re-run and confirmed passing the conductor). Wired into scripts/pre_build_verification.sh as invariant 58/58 (MD-EXPORT-TWINS-COMMITTABLE), mirror

ATM ID	Type	Status	Severity	Description
				the sibling CM-GITIGNORE-SWALLOW-GUARD (BOB-212) pattern exactly — BLOCKING, not advisory, since it is a pure tree-scan needing live stack. REMAINING WORK TO FULLY CLOSE THIS ITEM: design and implement the criterion residual — a mechanism distinguishing “a legitimate QA-evidence file lost to a secret-shield gitignore pattern” from “an actual secret correctly caught by that pattern” — without narrowing the existing credential-protecting globs. Until then this item stays Queued; criteria 1/2/4 are genuinely done, criterion 3 is genuinely partial. EVIDENCE: docs/qa/BOB-219/closure_evidence_20260925.md (full command transcripts: before/after git check-ignore, the wide-rescue safety experiment, 7-arm test output, standalone gate invocation against the real repo). WHAT, both conductor-verified with control needles: CM-SCRIPT-DOCS-SYNC occurrences scripts/pre_build_verification.sh = 0 (control needle CM-MARKDOWN-EXPORT-SYNC = 6, so instrument sees). §11.4.44 revision-header enforcement likewise 0 hits (control needle 3). Neither §11.4.18’s companion-doc mandate nor §11.4.44’s revision header is enforced by any invariant. THE PERVERSE CONSEQUENCE, MEASURED: a new script with NO companion doc passes the gate. Writing the companion doc that §11.4.18 MANDATES creates docs/scripts/.md, which invariant 16 (CM-MARKDOWN-EXPORT-SYNC) then requires to have .html and .pdf twins — and their absence is a BLOCKING failure. This session hit it exactly docs/scripts/test_gitignore_swallow_is_loud. {html,pdf} are missing and are two of invariant 16’s six violations. So the gate’s incentive gradient points AWAY from the constitution: complying with §11.4.18 is punished, ignoring is free. That is worse than an ungated rule — a rule the machinery actively discourages. §11.4.227 IS THE GOVERNING ANCHOR: a na
BOB-223	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				gate that exists only in prose is gate debt, and ledger of unimplemented CM-* names must be monotone-decreasing. CM-SCRIPT-DOCS-SYN named in §11.4.18's own text and implemented nowhere — one concrete row of exactly the 5 unimplemented population §11.4.227 was meant to shrink. ACCEPTANCE: (1) either CM-SCRIPT-DOCS-SYNC is implemented, or it is registered as a deferral pointing at a tracked §11.4.197 item; (2) silent absence is what §11.4.227 forbids; (3) so for §11.4.44's header check; (4) whichever is chosen, the incentive inversion is closed: it must never be cheaper to skip a mandated doc than to write one — if invariant 16 will demand twin docs, the doc-creation path must produce them, or invariant 16 must scope out docs/scripts until they can; (5) a paired §1.1 mutation per gate implemented. HONEST NOTE ON SCOPE: this item does not argue §11.4.18 should be enforced immediately — that is an operator call about debt priority (§11.4.66). It argues the CURRENT state is incoherent: an unenforced mandate whose observance triggers a different gate's failure. Either enforce both ends or neither. DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight while explaining why the new file broke invariant 16. Not by the automated QA regime. BOB-227 measured 3 of 4 artifacts of the CM-LAN-ROUTES-AUTHENTICATED gate untracked; criterion 1 (commit them) was already resolved by an unrelated prior commit before this session, closed 2026-09-23. Criterion 2 was NOT addressed: 'a gate asserting that every executable a pre-build invariant invokes is tracked' — a general mechanism preventing the whole CLASS of defect (a pre-build gate's own implementation files shipping untracked, invisible to a fresh clone, no committed baseline for round-over-round diffs) from recurring for ANY future gate, not merely this one. ACCEPTANCE: (1) enumerate every file path
BOB-231	Task	Queued	Major	

ATM ID	Type	Status	Severity	Description
BOB-235	Bug	Operator-blocked	—	scripts/pre_build_verification.sh invokes (via bash/timeout/python3 calls to scripts under scripts/pre_build/, plus every tests/pre_build/ and tests/hooks/.sh it runs) – likely via a static grep/parse of pre_build_verification.sh itself, runtime trace; (2) assert every one of those p is git-tracked (git ls-files –error-unmatch); (3) wire this as a new pre-build invariant so a fu untracked gate implementation is caught immediately, not discovered independently weeks later; (4) a RED test creating an untrac fake gate-invocation target and asserting the check fails on it, GREEN once the mechanism exists and the fake target is either removed o tracked. Live kinozal credential test reports authenticated=False, status='empty', error=' (found 2026-09-23). ROOT CAUSES (docs/qa/BOB-235/investigation_20260923.md): (A) operator/environment: kinozal.tv publishes A record 127.0.0.1 (host, container, DoH Cloudflare+Google all agree; rutracker.org resolves normally as control), KINOZAL_MIRRORS is empty so search.py fal back to kinozal.tv; the live mirror kinozal.gu answers every non-browser client with a Cloudflare 403 'Just a moment' challenge (log POST, curl, aiohttp, operator cookies with two user-agents; no cf_clearance cookie; KINOZAL_COOKIES is loaded into the contain but no code reads it). (B) code defect FIXED under TDD (RED 2 failed/1 passed, GREEN 19 passed): _search_kinozal's except branch only logged and stashed no diagnostic, so the chip read empty with error=None; it now stashes error_type + 'Kinozal request failed: '. Needs start.sh –reload-python to be live. Credential validity is UNKNOWN (no request ever reach login endpoint) - do not rotate credentials on evidence. Operator-Block-Details: WHAT: cho (1) reachable domain - set KINOZAL_MIRRORS=https://kinozal.guru + ./

ATM ID	Type	Status	Severity	Description
BOB-237	Bug	Queued	high	start.sh –recreate, or change the roster prima in trackers.py; (2) how to pass the Cloudflare challenge - FlareSolvrr (not deployed), cf_clearance cookie + code that actually reads KINOZAL_COOKIES, Jackett’s kinozal indexer mark kinozal operator-blocked; (3) optionally export a kinozal-only cookie file. WHY: every agent-reachable path was exhausted - DNS, mirrors, login POST, cookies, env propagation measured. UNBLOCK CONDITION: the live test authenticates or the operator marks kinozal unsupported. WHO: repository operator. WHAT: CM-GATE-LEDGER-RATCHET (§11.4.227(A)) FAILED the pre-build sweep during this session’s batch-15 commit attempt: unimplemented=414 exceeds the checked-in baseline=403 in constitution/scripts/gates/gate_ledger_baseline.txt. This is a HARD, blocked FAIL (unlike the other gate findings surfaced the same sweep, which are all explicitly ADVISORY/non-blocking per §11.4.234). INVESTIGATION (§11.4.102): the four newest anchors this session’s constitution pull brought in (§11.4.268 tamper-evident evidence chain, §11.4.269 critic/consensus advisory-only ban, §11.4.270 dependency-existence-verdict register §11.4.271 waiver mechanism) were checked FIRST as the likely cause, since they are the most recently landed (2026-08-26) and each names several recommended mechanism gates in its own text. RULED OUT: every one of those gate names (CM-CRITIC-CONSENSUS-ADVISORY-ONLY, CM-DEPENDENCY-EXISTENCE-VERDICT-REGISTER, CM-EVIDENCE-CHAIN-ANCHOR-CATCHES-RECOMPUTE-AND-TRUNCATION, CM-EVIDENCE-CHAIN-DELETE-REORDER-DETECT, CM-EVIDENCE-CHAIN-INCOMPLETE-VERIFICATION-REFUSES, CM-WAIVER-ROSTERED-EXPIRY-TRACKED) is already correctly registered DEFERRED against an OWED-GATE-NNN tracked item, and each anchor’s own §11.4.227(B) propagation gate is

ATM ID	Type	Status	Severity	Description
				IMPLEMENTED. So this batch of new anchors NOT introduce the ratchet violation. The full ledger run (docs/qa/BOB-237/unimplemented_gates_snapshot_20260925.tx 414 lines, captured verbatim from bash constitution/scripts/gates/cm_gate_ledger_ratchet.sh) shows the total current state is 414 UNIMPLEMENTED + 121 IMPLEMENTED + 73 DEFERRED. The checked baseline (403) was last legitimately bumped by commit 34e42f0 (“cite the §11.4.209 gate rename bump baseline 397->403”) — some point after that, the unimplemented count grew to 414 (more) without a matching baseline bump or deferral registration for the newly-named gate across one or more subsequent anchor-landing rounds in the constitution submodule. This is accumulated UNIVERSAL governance-corpus debt, not something introduced by any boba-edit in this session (confirmed: git diff eba38e8..HEAD -- constitution/scripts/gate_gate_ledger_baseline.txt inside the constitution submodule is empty — the baseline file itself never touched across the two merges this session performed). WHY DEFERRED RATHER THAN FIXED HERE: identifying which of the 414 unimplemented names are the specific 11 that pushed past baseline, and then either implementing 11 gates’ worth of real UNIVERSAL gate-code or registering 11 proper OWED-GATE NNN deferrals with tracked-item citations, is itself real engineering work scoped to the constitution submodule (universal, not project-specific per §11.4.17) — it is not a boba-project and doing it hastily mid-session, unrelated to session’s actual mandate (Jackett incorporation backlog triage), risks exactly the kind of rushed unreviewed governance-corpus edit §11.4.209 Fable-xhigh review requirement exists to prevent. ACCEPTANCE: (1) a future session (or the constitution’s own maintainers) triages the 414-line snapshot against the 403 baseline to identify the specific unaccounted gate names

ATM ID	Type	Status	Severity	Description
				each is either implemented, registered as a proper OWED-GATE-NNN deferral, or (if genuinely retired) cited in constitution/scripts/gates/gate_ledger_removals.tsv; (3) the constitution submodule's baseline is re-bumped with a citation once accounted, following the same pattern as commit 34e42f0; (4) boba's pinned constitution pointer is then re-synced with this item closed with the passing CM-GATE-LEDGER-RATCHET re-run as evidence. DISPOSITION FOR THIS SESSION'S COMMIT: §11.4.234(D) (the commit/push mechanism MUST always be able to complete; a failing gate is skippable ONLY via an explicit recorded deferral flag, with the skip recorded in the commit message so the debt stays tracked, never forgotten), this session's batch-15 commit uses the sanctioned skip (BOBA_SYNC_SKIP_CI=1), citing this tracked item BOB-237, rather than silently bypassing or spending unbounded effort resolving an inherited universal-corpus debt mid-session. DISCOVERY CHANNEL (§11.4.238) found by boba's own pre-build sweep (scripts/pre_build_verification.sh, invariant 38/57) during a routine session commit — the automated gate itself is the discoverer, exactly as §11.4.238 requires.